工作汇报

张鹤

2020.5.8

1.阅读文献

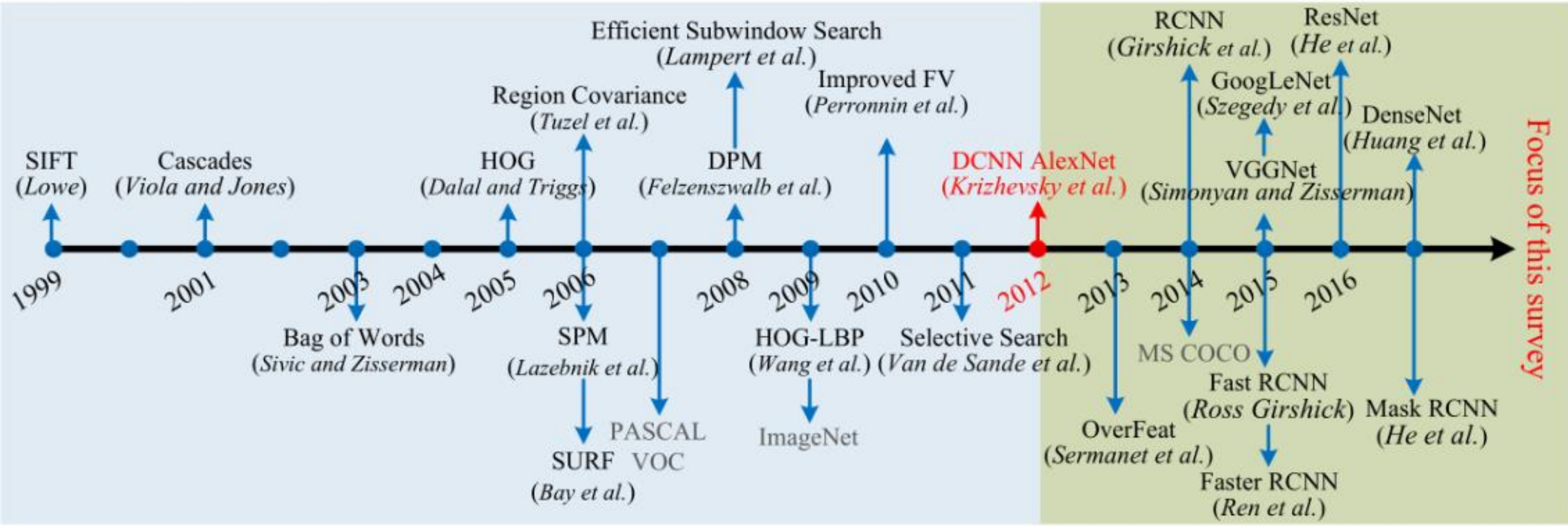
2.模式识别实验

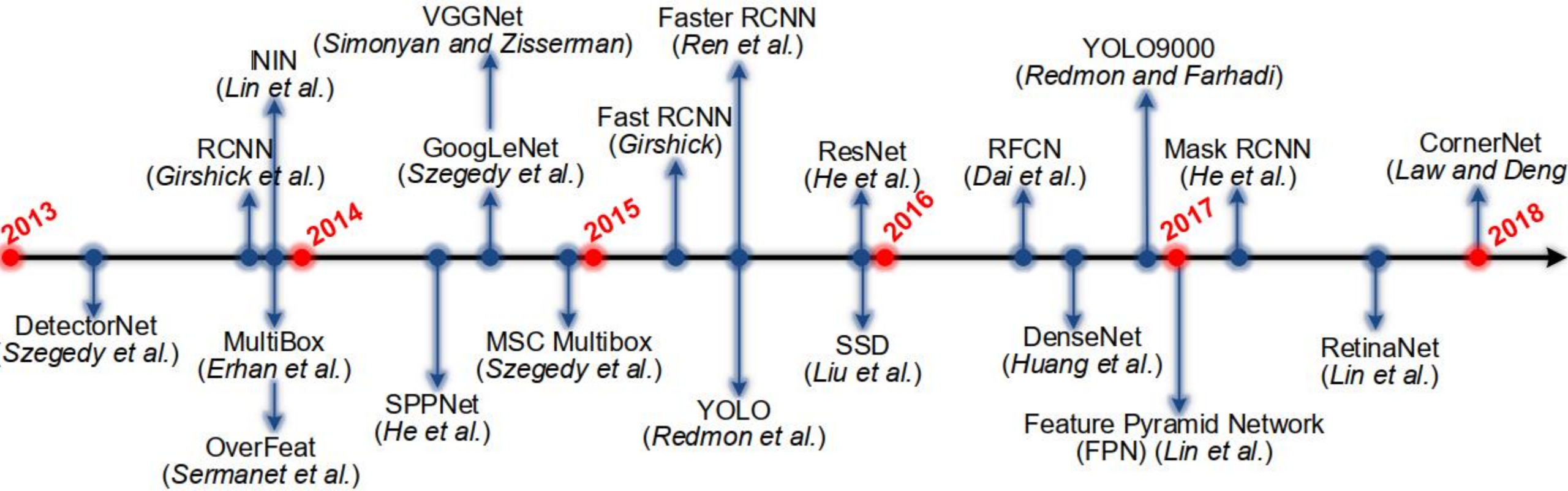
3.模式识别扩展实验

Deep Learning for Generic Object Detection: A Survey

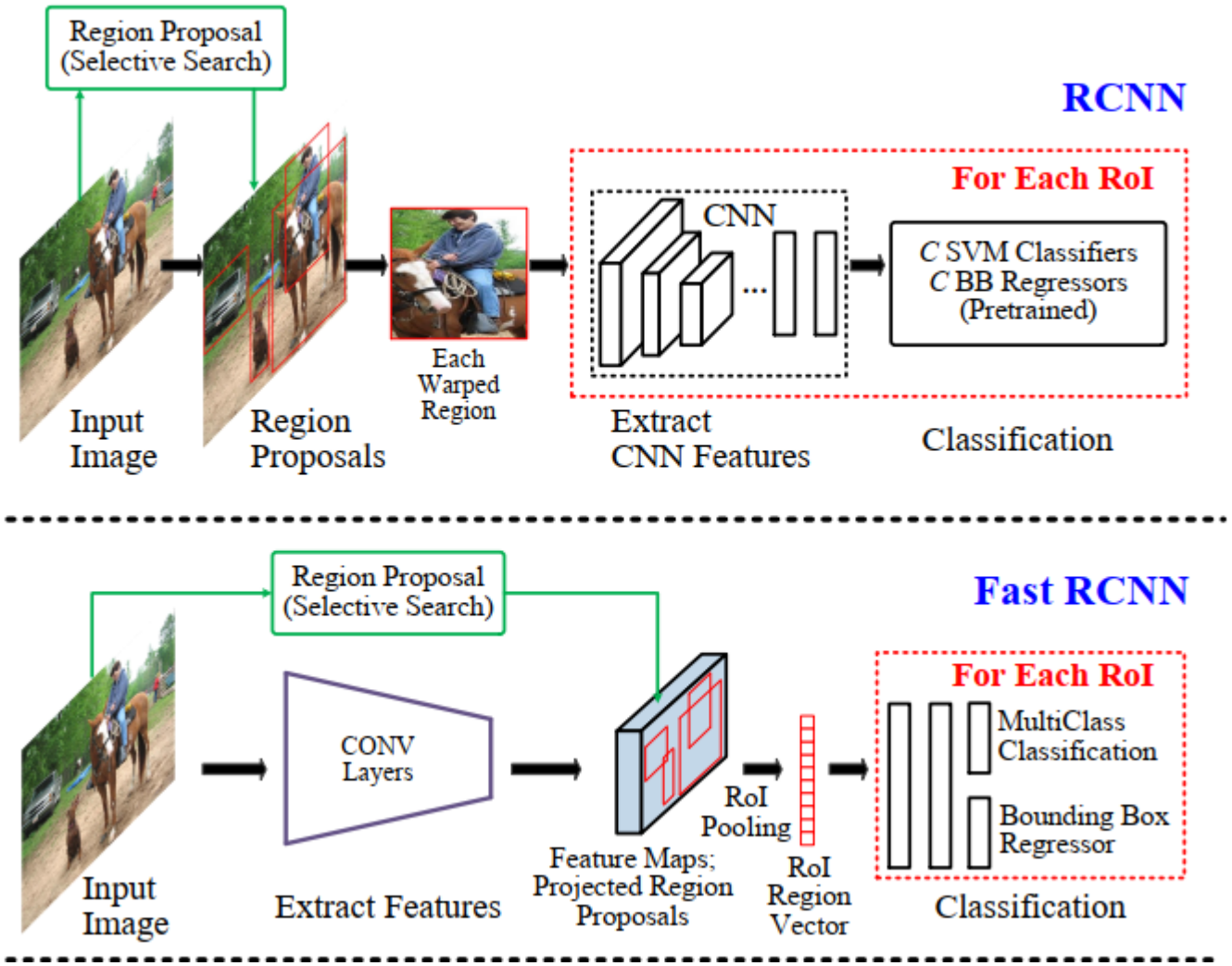
Li Liu1,2·Wanli Ouyang3·Xiaogang Wang4·Paul Fieguth5·Jie Chen2·Xinwang Liu1·Matti Pietikäinen

https://github.com/hoya012/deep_learning_object_detection#2014


SIFT
(Lowe)
Cascades
(Viola and Jones)
HOG
(Dalal and Triggs)
Region Covariance
(Tuzel et al.)
Efficient Subwindow Search
(Lampert et al.)
DPM
(Felzenszwalb et al.)
Improved FV
(Perronnin et al.)
DCNN AlexNet
(Krizhevsky et al.)
RCNN
(Girshick et al.)
GoogLeNet
(Szegedy et al.)
VGGNet
(Simonyan and Zisserman)
ResNet
(He et al.)
DenseNet
(Huang et al.)
Focus of this survey
1999
2001
2003
2004
2005
2006
2008
2009
2010
2011
2012
2013
2014
2015
2016
Bag of Words
(Sivic and Zisserman)
SPM
(Lazebnik et al.)
SURF
(Bay et al.)
PASCAL VOC
HOG-LBP
(Wang et al.)
ImageNet
Selective Search
(Van de Sande et al.)
OverFeat
(Sermanet et al.)
MS COCO
Fast RCNN
(Ross Girshick)
Faster RCNN
(Ren et al.)
Mask RCNN
(He et al.)


2013
2014
2015
2016
2017
2018
DetectorNet
(Szegedy et al.)
RCNN
(Girshick et al.)
NIN
(Lin et al.)
MultiBox
(Erhan et al.)
OverFeat
(Sermanet et al.)
VGGNet
(Simonyan and Zisserman)
GoogLeNet
(Szegedy et al.)
SPPNet
(He et al.)
Fast RCNN
(Girshick)
MSC Multibox
(Szegedy et al.)
Faster RCNN
(Ren et al.)
YOLO
(Redmon et al.)
ResNet
(He et al.)
SSD
(Liu et al.)
RFCN
(Dai et al.)
DenseNet
(Huang et al.)
YOLO9000
(Redmon and Farhadi)
Feature Pyramid Network
(FPN) (Lin et al.)
Mask RCNN
(He et al.)
RetinaNet
(Lin et al.)
CornerNet
(Law and Deng

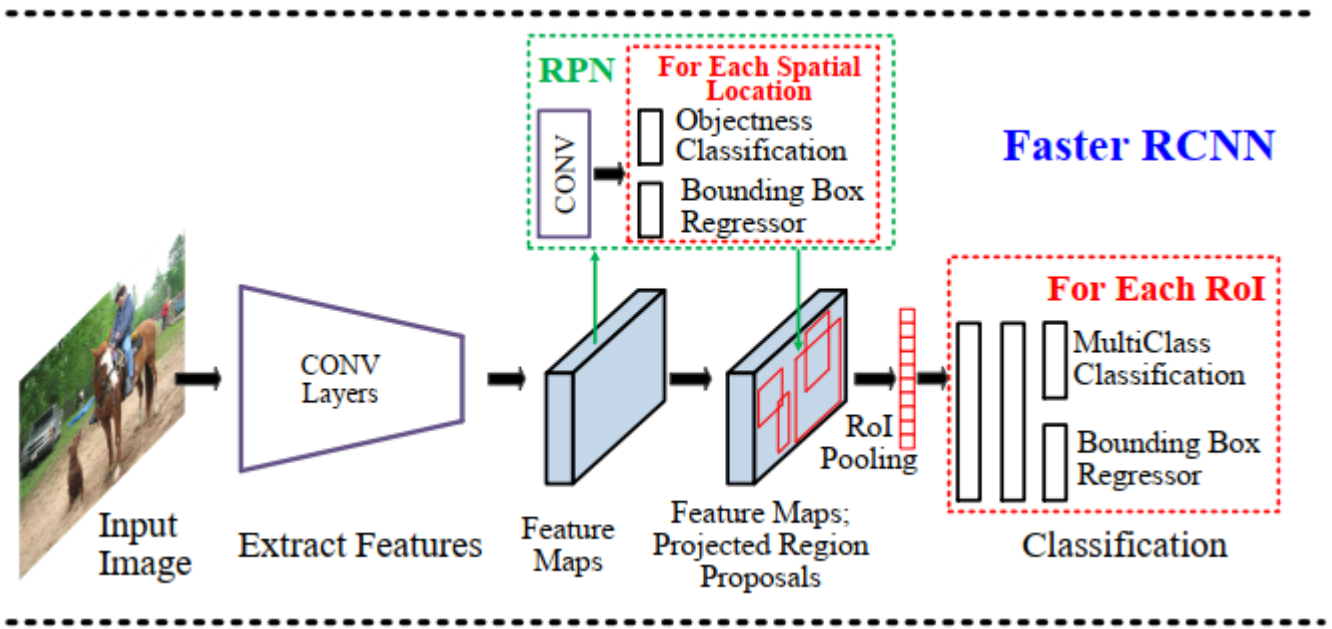
两阶段：基于区域



R-CNN (Selective Search + CNN + SVM)

Fast R-CNN (Selective Search + CNN + ROI)

两阶段：基于区域



Faster R-CNN (RPN + CNN + ROI)

一阶段：基于单次目标

一阶段的网络，整个生成备选框的过程一体化的实现。代表有

YOLO系列和SSD（single shot detector）

DetectorNet

OverFeat

YOLO

YOLOv2 and YOLO9000

SSD(Single Shot Detector)

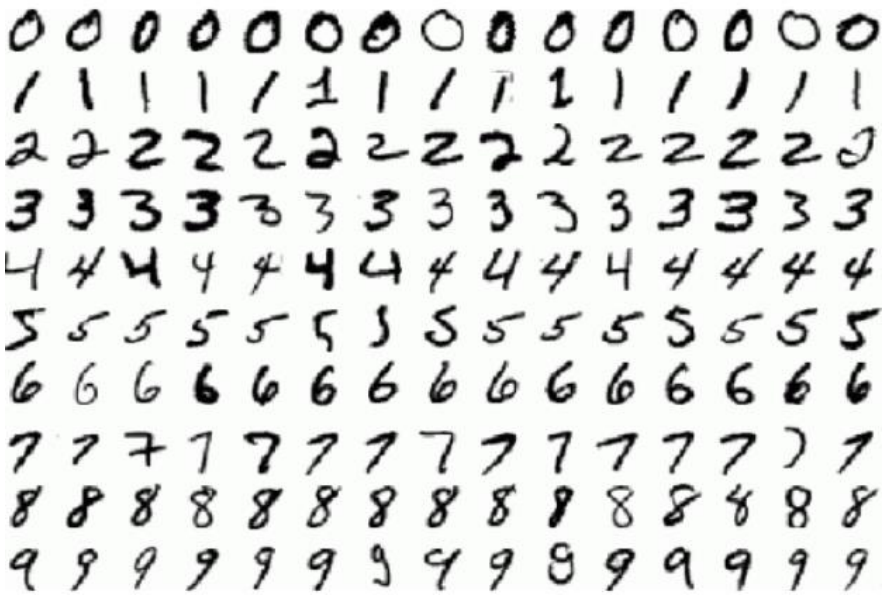
No.	DCNN Architecture	#Paras ($\times 10^6$)	#Layers (CONV+FC)	Test Error (Top 5)	First Used In	Highlights
1	AlexNet [141]	57	$5+2$	15.3%	[85]	The first DCNN found effective for ImageNet classification; the historical turning point from hand-crafted features to CNN; Winning the ILSVRC2012 Image classification competition.
2	ZFNet (fast) [303]	58	$5+2$	14.8%	[99]	Similar to AlexNet, different in stride for convolution, filter size, and number of filters for some layers.
3	OverFeat [239]	140	$6+2$	13.6%	[239]	Similar to AlexNet, different in stride for convolution, filter size, and number of filters for some layers.
4	VGGNet [248]	134	$13+2$	6.8%	[84]	Increasing network depth significantly by stacking 3×3 convolution filters and increasing the network depth step by step.
5	GoogLeNet [263]	6	22	6.7%	[263]	Use Inception module, which uses multiple branches of convolutional layers with different filter sizes and then concatenates feature maps produced by these branches. The first inclusion of bottleneck structure and global average pooling.
6	Inception v2 [125]	12	31	4.8%	[112]	Faster training with the introduce of Batch Normalization.
7	Inception v3 [264]	22	47	3.6%		Inclusion of separable convolution and spatial resolution reduction.
8	YOLONet [227]	64	$24+1$	–	[227]	A network inspired by GoogLeNet used in YOLO detector.
9	ResNet50 [101]	23.4	49	3.6%	[101]	With identity mapping, substantially deeper networks can be learned.
10	ResNet101 [101]	42	100	(ResNets)	[101]	Requires fewer parameters than VGG by using the global average pooling and bottleneck introduced in GoogLeNet.

11	InceptionResNet v1 [265]	21	87	3.1% (Ensemble)		Combination of identity mapping and Inception module, with similar computational cost of Inception v3, but faster training process.
12	InceptionResNet v2 [265]	30	95		[120]	A costlier residual version of Inception, with significantly improved recognition performance.
13	Inception v4 [265]	41	75			An Inception variant without residual connections, with roughly the same recognition performance as InceptionResNet v2, but significantly slower.
14	ResNeXt [291]	23	49	3.0%	[291]	Repeating a building block that aggregates a set of transformations with the same topology.
15	DenseNet201 [118]	18	200	–	[321]	Concatenate each layer with every other layer in a feed forward fashion. Alleviate the vanishing gradient problem, encourage feature reuse, reduction in number of parameters.
16	DarkNet [226]	20	19	–	[226]	Similar to VGGNet, but with significantly fewer parameters.
17	MobileNet [112]	3.2	27 + 1	–	[112]	Light weight deep CNNs using depth-wise separable convolutions.
18	SE ResNet [115]	26	50	2.3% (SENets)	[115]	Channel-wise attention by a novel block called *Squeeze and Excitation*. Complementary to existing backbone CNNs.

架构演变的趋势是,网络越来越深入:AlexNet由8层,VGGNet 16层,以及最近ResNet和DenseNet都超过了100层, 特别是 VGGNet和GoogLeNet，这表明,增加深度可以提高深层网络的表征能力。

有趣的是，从表2中可以观察到，AlexNet、OverFeat、ZFNet和VGGNet等网络的参数数量非常庞大，尽管只有几层，因为大部分的参数来自FC层。因此，像Inception、ResNet和DenseNet这样的新网络虽然具有非常大的网络深度，但是由于避免使用FC层，它们的参数要少得多

MNIST数据集



60000训练+10000测试

算法思路

1. 下载并读入数据集
2. 将要识别的图片转为灰度图，并且转化为28*28矩阵
3. 将28*28的矩阵转换成1维矩阵（也就是把第2,3,4,5....行矩阵接入到第一行的后面）
4. 将标签用独热编码（one-hot）表示,数字1对应的矩阵就是[0,1,0,0,0,0,0,0,0,0]
5. softmax回归预测图片是哪个数字的概率
6. 用交叉熵和梯度下降法训练参数
7. 计算准确率，进行交叉验证
8. 改变学习率、训练轮数、batch size大小观察对结果影响

学习率=0.01，batch size=128，epoch=100

```
Eppoch: 000/100 cost: 1.281495290 train_acc: 0.836 test_acc: 0.845
Eppoch: 005/100 cost: 0.469517576 train_acc: 0.882 test_acc: 0.891
Eppoch: 010/100 cost: 0.402460976 train_acc: 0.893 test_acc: 0.901
Eppoch: 015/100 cost: 0.373601074 train_acc: 0.899 test_acc: 0.907
Eppoch: 020/100 cost: 0.356141847 train_acc: 0.902 test_acc: 0.910
Eppoch: 025/100 cost: 0.344280148 train_acc: 0.905 test_acc: 0.912
Eppoch: 030/100 cost: 0.332898995 train_acc: 0.907 test_acc: 0.914
Eppoch: 035/100 cost: 0.326567471 train_acc: 0.909 test_acc: 0.915
Eppoch: 040/100 cost: 0.320394242 train_acc: 0.911 test_acc: 0.916
Eppoch: 045/100 cost: 0.316246868 train_acc: 0.912 test_acc: 0.917
Eppoch: 050/100 cost: 0.313470757 train_acc: 0.913 test_acc: 0.918
Eppoch: 055/100 cost: 0.307108887 train_acc: 0.915 test_acc: 0.918
Eppoch: 060/100 cost: 0.308242116 train_acc: 0.916 test_acc: 0.919
Eppoch: 065/100 cost: 0.303186604 train_acc: 0.916 test_acc: 0.919
Eppoch: 070/100 cost: 0.299111661 train_acc: 0.916 test_acc: 0.920
Eppoch: 075/100 cost: 0.296354776 train_acc: 0.917 test_acc: 0.920
Eppoch: 080/100 cost: 0.297743997 train_acc: 0.918 test_acc: 0.920
Eppoch: 085/100 cost: 0.297354974 train_acc: 0.918 test_acc: 0.920
Eppoch: 090/100 cost: 0.289803832 train_acc: 0.919 test_acc: 0.920
Eppoch: 095/100 cost: 0.291887972 train_acc: 0.919 test_acc: 0.920
```

```
TIME: 81.3735134601593
```

学习率=0.1，batch size=128，epoch=100

```
Eppoch: 000/100 cost: 0.568862970 train_acc: 0.891 test_acc: 0.900
Eppoch: 005/100 cost: 0.302776945 train_acc: 0.914 test_acc: 0.920
Eppoch: 010/100 cost: 0.282735155 train_acc: 0.919 test_acc: 0.921
Eppoch: 015/100 cost: 0.272860382 train_acc: 0.922 test_acc: 0.922
Eppoch: 020/100 cost: 0.265869325 train_acc: 0.925 test_acc: 0.925
Eppoch: 025/100 cost: 0.261983198 train_acc: 0.925 test_acc: 0.922
Eppoch: 030/100 cost: 0.257552409 train_acc: 0.927 test_acc: 0.924
Eppoch: 035/100 cost: 0.253489727 train_acc: 0.927 test_acc: 0.923
Eppoch: 040/100 cost: 0.252542518 train_acc: 0.928 test_acc: 0.923
Eppoch: 045/100 cost: 0.249614637 train_acc: 0.929 test_acc: 0.925
Eppoch: 050/100 cost: 0.249024641 train_acc: 0.930 test_acc: 0.925
Eppoch: 055/100 cost: 0.247106299 train_acc: 0.930 test_acc: 0.926
Eppoch: 060/100 cost: 0.244215391 train_acc: 0.930 test_acc: 0.925
Eppoch: 065/100 cost: 0.244144487 train_acc: 0.931 test_acc: 0.926
Eppoch: 070/100 cost: 0.244450188 train_acc: 0.932 test_acc: 0.925
Eppoch: 075/100 cost: 0.242815975 train_acc: 0.932 test_acc: 0.926
Eppoch: 080/100 cost: 0.244943878 train_acc: 0.932 test_acc: 0.926
Eppoch: 085/100 cost: 0.238570042 train_acc: 0.932 test_acc: 0.926
Eppoch: 090/100 cost: 0.241163947 train_acc: 0.933 test_acc: 0.925
Eppoch: 095/100 cost: 0.240509270 train_acc: 0.933 test_acc: 0.925
```

```
TIME: 79.44116163253784
```

batch size=10,128,256

```
Eppoch: 000/020 cost: 0.245723764 train_acc: 0.913 test_acc: 0.915
Eppoch: 005/020 cost: 0.187424685 train_acc: 0.923 test_acc: 0.919
Eppoch: 010/020 cost: 0.181113257 train_acc: 0.926 test_acc: 0.922
Eppoch: 015/020 cost: 0.177648153 train_acc: 0.927 test_acc: 0.919
Done.
TIME: 96.43818187713623
```

```
Eppoch: 000/020 cost: 0.716046705 train_acc: 0.877 test_acc: 0.885
Eppoch: 005/020 cost: 0.336735850 train_acc: 0.907 test_acc: 0.912
Eppoch: 010/020 cost: 0.309698636 train_acc: 0.914 test_acc: 0.918
Eppoch: 015/020 cost: 0.294023043 train_acc: 0.918 test_acc: 0.919
Done.
TIME: 11.037804365158081
```

```
Eppoch: 000/020 cost: 0.569299626 train_acc: 0.890 test_acc: 0.898
Eppoch: 005/020 cost: 0.303431625 train_acc: 0.913 test_acc: 0.917
Eppoch: 010/020 cost: 0.282022437 train_acc: 0.920 test_acc: 0.922
Eppoch: 015/020 cost: 0.272810050 train_acc: 0.923 test_acc: 0.924
Done.
TIME: 15.26936674118042
```

Batch数不同的影响

1）batch数太小，而类别又比较多的时候，真的可能会导致loss函数震荡而不收敛，尤其是在网络比较复杂的时候。
2）随着batchsize增大，处理相同的数据量的速度越快。
3）随着batchsize增大，达到相同精度所需要的epoch数量越来越多。
4）由于上述两种因素的矛盾， Batch_Size 增大到某个时候，达到时间上的最优。
5）由于最终收敛精度会陷入不同的局部极值，因此 Batch_Size 增大到某些时候，达到最终收敛精度上的最优。
6）过大的batchsize的结果是网络很容易收敛到一些不好的局部最优点。同样太小的batch也存在一些问题，比如训练速度很慢，训练不容易收敛等。
7）具体的batch size的选取和训练集的样本数目相关。

K折交叉验证

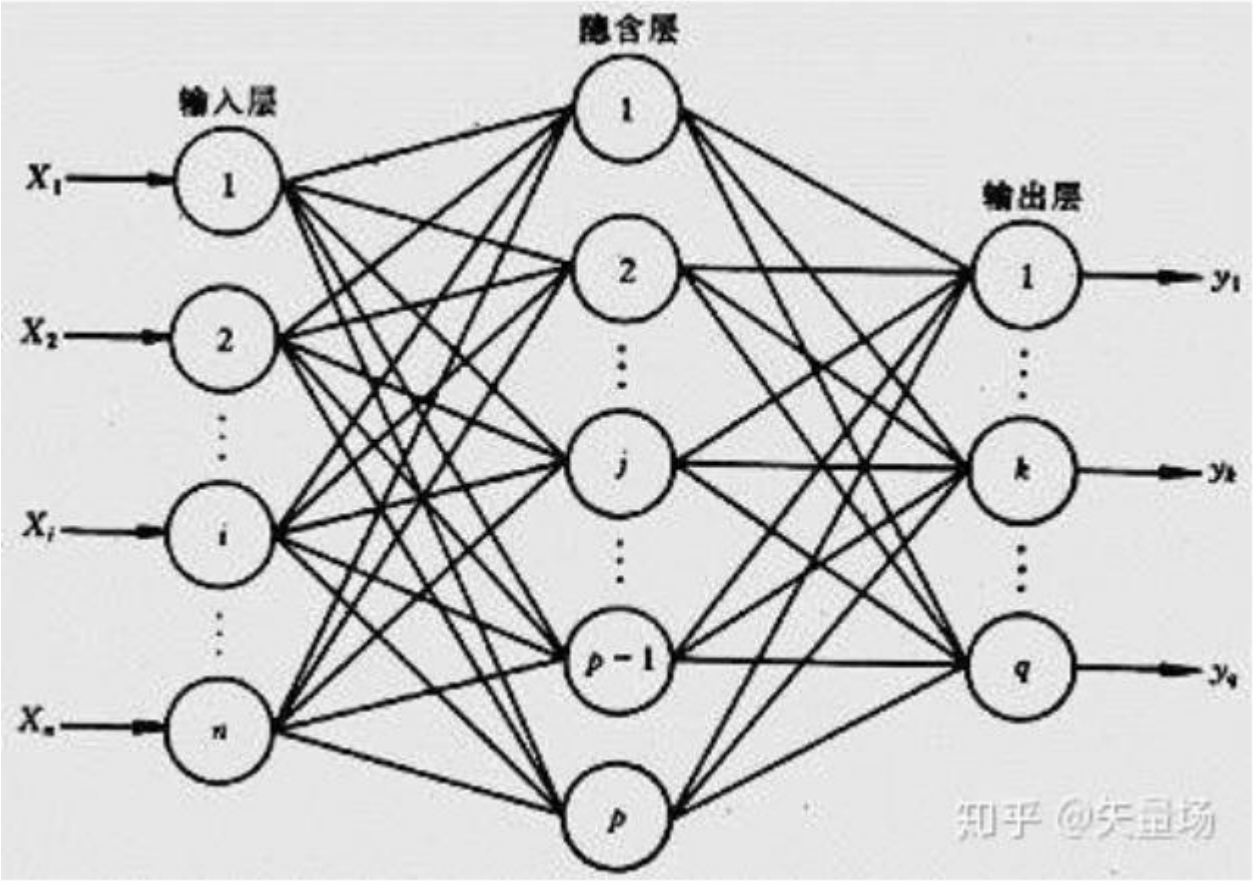
```
第 1 轮交叉验证的准确率为 0.8921
第 2 轮交叉验证的准确率为 0.8860
第 3 轮交叉验证的准确率为 0.8943
第 4 轮交叉验证的准确率为 0.8898
平均交叉验证准确率为 0.8905
```

进行4折交叉验证。每次选取四分之三的数据作为训练集，剩下的作为验证集，求出每轮交叉验证的准确率、偏差、方差等数据，学习率设置为0.1，设置轮数20轮，batch size大小为128。对于测试集用每轮交叉验证的模型预测一遍，最后归为最大值所属类别。

神经网络算法

输入层：784
隐含层：256（[b, 784] => [b, 256]）
128（[b, 256] => [b, 128]）
64（[b, 128] => [b, 64]）
32（[b, 64] => [b, 32]）
输出层：10

隐含层使用relu激活函数



不同的batch size

			train		validation		test		
batchsize	隐藏层	epoch	loss	accuracy	loss	accuracy	loss	accuracy	time
10	256+128+64+32（relu）	10	0.0316	0.991	0.1381	0.9735	0.1156	0.9755	136.8293934
128	256+128+64+32（relu）	10	0.0171	0.9938	0.0857	0.9795	0.0871	0.9777	21.48318934
256	256+128+64+32（relu）	10	0.0171	0.9945	0.0834	0.9803	0.0687	0.9807	19.5258038
1000	256+128+64+32（relu）	10	0.0453	0.987	0.1042	0.9712	0.0921	0.9733	13.25937486

loss	accuracy
0.0748	0.9802

不同的激活函数

batchsize	隐藏层	epoch	loss	accuracy	loss	accuracy	loss	accuracy	time
128	256+128+64+32（sigmoid）	10	0.0449	0.9887	0.1035	0.9712	0.0941	0.9728	21.93003249
256	256+128+64+32（sigmoid）	10	0.0701	0.9823	0.1346	0.9655	0.1066	0.9715	18.68892908
1000	256+128+64+32（sigmoid）	10	0.2245	0.9534	0.2352	0.949	0.2307	0.9487	14.3109045
128	256+128+64+32（tanh）	10	0.0202	0.9941	0.0929	0.976	0.081	0.977	25.2592988
256	256+128+64+32（tanh）	10	0.0187	0.9949	0.0939	0.9745	0.0837	0.9744	20.80274415
1000	256+128+64+32（tanh）	10	0.0571	0.9847	0.1023	0.9708	0.092	0.9724	12.97104859

两个隐含层

128	256+64(relu)	10	0.0135	0.9959	0.0805	0.9803	0.0811	0.9793	19.83192134	
256	256+64(relu)	10	0.0142	0.9966	0.0844	0.9783	0.0691	0.9797	15.95937061	过拟合
1000	256+64(relu)	10	0.0523	0.9854	0.0967	0.9732	0.0863	0.974	12.00048351	

Thank You!